

Cross-Platform Mobile App Development

Week 7 — Flutter & Dart (Part 1): Fundamentals & Widget Tree

Instructor: Nguyen Thanh Tuan, PhD

Institution: Faculty of Computer Science, VKU

Today's Agenda

1. **The Flutter Philosophy & Architecture** — "Everything is a Widget", AOT vs. Bridge, Impeller Engine
2. **Modern Dart 3 for Mobile Engineers** — Null Safety, Records, Patterns, `const` Optimization, Async
3. **The 3 Trees in Flutter** — Widget Tree, Element Tree, RenderObject Tree, and Key Mechanics
4. **Stateless vs. Stateful Widgets** — Complete State Lifecycle & Memory Management
5. **Layout Composition & Material 3** — Row, Column, Stack, Expanded, ListView.builder, SafeArea
6. **Mini-Project 3 Kickoff** — On-Device OCR Expense Tracker (ML Kit, Heuristic Parser, Local SQLite)
7. **Hands-on Lab Exercise** — Building a Material 3 Responsive Expense Card

Part 1: The Flutter Philosophy & Architecture

The Flutter Philosophy: "Everything is a Widget"

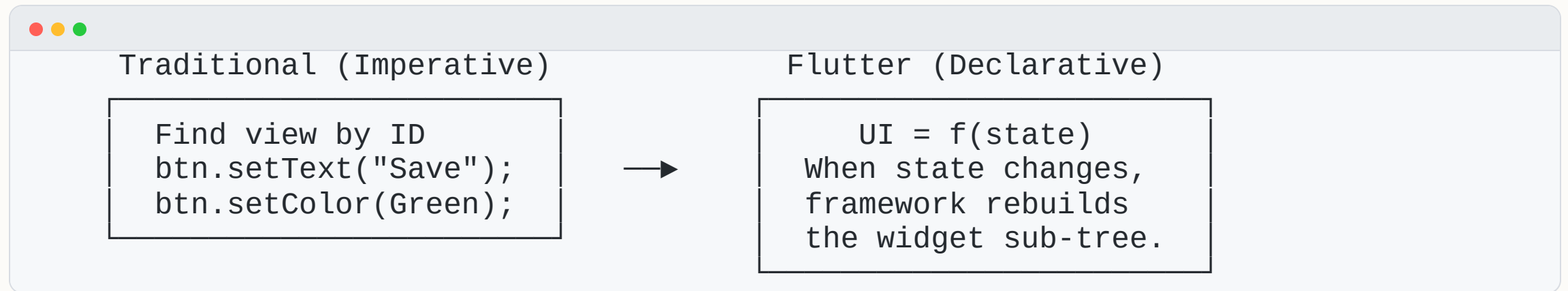
In Flutter, **widgets** are the universal building blocks of the entire user interface:

- **Structural Elements:** Buttons, text labels, icons, image views, input fields.
- **Layout Models:** Padding, alignment, rows, columns, grids, aspect ratios.
- **Styling & Theming:** Color palettes, typography styles, decorations, theme tokens.
- **Interactivity & Animation:** Gesture detectors, tap listeners, animated transitions.

In Flutter, there are no separate XML or CSS files. Layout, styling, and UI logic are composed together using a unified Dart mental model.

Declarative UI: $UI = f(state)$

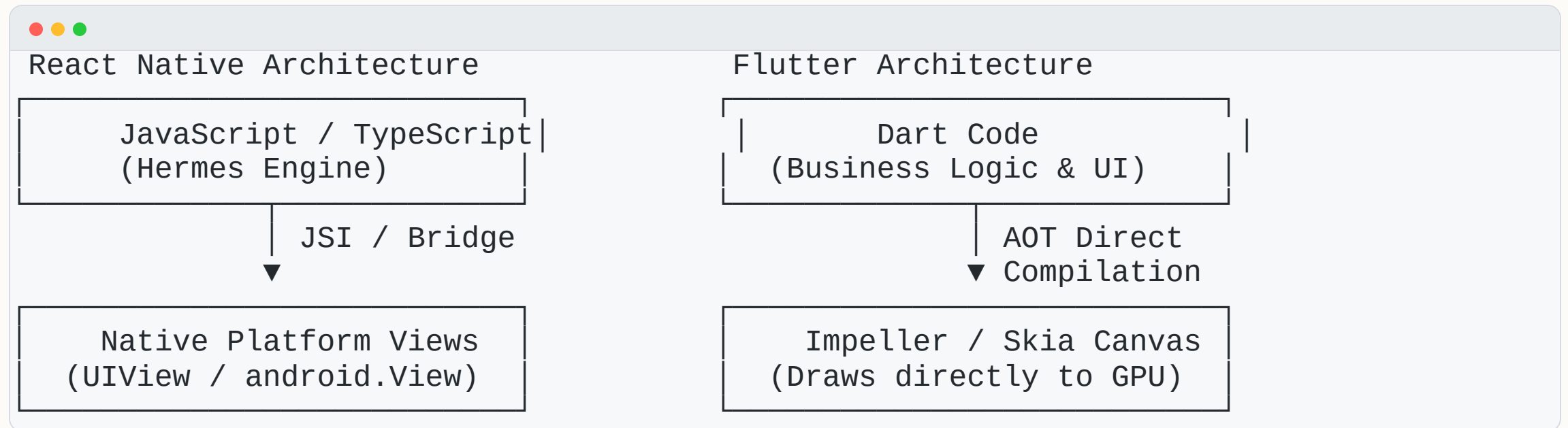
Flutter fundamentally departs from traditional imperative UI toolkits:



- **Imperative:** You manually mutate existing view instances step-by-step.
- **Declarative:** You describe how the UI looks for any given state. When state changes, Flutter reconstructs the sub-tree.

Direct Native Compilation (Zero Bridge)

Unlike hybrid WebViews or the React Native bridge, Flutter compiles directly to machine code:



- **AOT (Ahead-of-Time):** Compiles Dart directly into **ARM64 / x86_64** native machine code for production. Zero bridge serialization overhead.
- **JIT (Just-in-Time):** Used during development to power sub-second **Stateful Hot Reload**.

Flutter Architectural Layers

Framework (Dart)

- Material / Cupertino (Platform Design Systems)
- Widgets (Composition Layer: Text, Row, Stack, Container)
- Rendering (Layout calculation, Hit testing, Paint commands)

Engine (C / C++)

- Impeller / Skia (High-performance 2D rasterization graphics)
- Dart VM & Runtime (Garbage collection, Isolates, Event loop)
- Text Layout (HarfBuzz, Libunibreak, ICU typography engine)

Embedder (Platform-Specific Shell)

- Android (NDK/JNI) | iOS (Metal/UIKit) | Web / macOS / Windows

The framework is completely swappable and extensible in user-space Dart code.

Rendering Evolution: Skia vs. Impeller

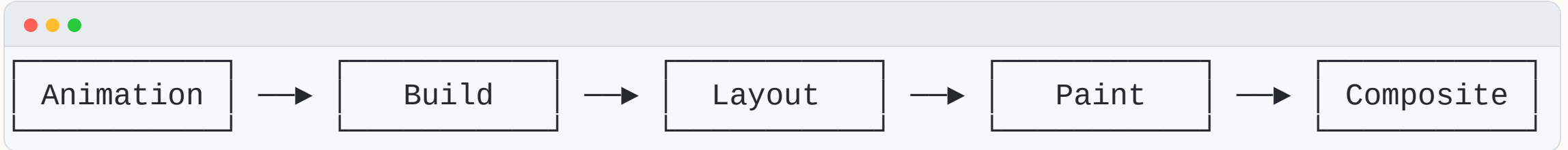
For years, Flutter relied on Google's **Skia** 2D graphics library. In modern Flutter, **Impeller** is the default renderer for iOS and Android:

Feature	Legacy Skia Engine	Modern Impeller Engine
Shader Compilation	Runtime JIT compilation	Pre-compiled at build time
First-Frame Jank	⚠ Noticeable shader compilation stutter	✅ Zero shader jank (smooth first load)
Graphics Backend	OpenGL ES / Vulkan	Metal (iOS) & Vulkan (Android)
Target Frame Rate	60 FPS	Rock-solid 120 FPS ProMotion

Why it matters: Impeller guarantees frame delivery within the strict **8.33ms** (120Hz) or **16.6ms** (60Hz) hardware budget.

The 120Hz Frame Pipeline

Every single frame in Flutter flows through 5 deterministic stages:



1. **Animation:** VSync signal ticks animation controllers and physics simulations.
2. **Build:** Flutter invokes `build()` only on widgets marked "dirty" by `setState()`.
3. **Layout:** Computes constraints from parent down to child; sizes go child up to parent.
4. **Paint:** Render objects record draw commands (`drawRect` , `drawPath` , `drawImage`).
5. **Compositing:** Layers are uploaded to GPU buffers for hardware rasterization.

Part 2: Modern Dart 3 for Mobile Engineers

Why Dart for Flutter?

Google chose Dart for Flutter because of its unique hybrid runtime:

- **Stateful Hot Reload (JIT):** Injects updated source code into the running VM in ~400ms without restarting the app or resetting UI state.
- **Native Speed (AOT):** Production builds compile to native binary with predictable performance and no JIT warm-up lag.
- **Single-Threaded Event Loop with Isolates:** Fast garbage collection optimized for short-lived, immutable UI objects.
- **Type Safety & Soundness:** Dart's type system is 100% sound — if a variable is `int`, it can **never** be `null` or a string at runtime.

```
# Verify your local Dart 3 environment
flutter --version
dart --version
```

Sound Null Safety in Dart 3

Dart prevents runtime `NullPointerException` bugs at compile time:

```
// 1. Non-nullable type (cannot be null)
String merchantName = "Highlands Coffee";

// 2. Nullable type (explicitly marked with '?')
double? receiptDiscount; // Defaults to null

// 3. Null-aware fallback operator '??'
double finalDiscount = receiptDiscount ?? 0.0;

// 4. Null-aware assignment '??='
receiptDiscount ??= 5.0; // Assigns only if currently null

// 5. Null-safe navigation '?.'
String? upperMerchant = merchantName?.toUpperCase();

// 6. Null assertion operator '!' (Throws error if null!)
double forcedValue = receiptDiscount!;
```

Dart 3 Records: Lightweight Anonymous Tuples

Dart 3 introduces **Records** for grouping heterogeneous values without boilerplate classes:

```
// Return multiple values from a receipt parser function
(String merchant, double total, DateTime date) parseReceipt(String rawText) {
  return ("Co.op Mart", 245000.0, DateTime.now());
}

void processReceipt() {
  // 1. Positional access ($1, $2, $3)
  final result = parseReceipt("raw OCR stream");
  print("Store: ${result.$1}, Total: ${result.$2}");

  // 2. Destructuring assignment (cleaner syntax)
  final (merchant, total, date) = parseReceipt("raw OCR stream");
  print("Extracted $total VND from $merchant on $date");
}
```

Records are **value-typed**, immutable, and provide compile-time type safety.

Pattern Matching & Switch Expressions

Dart 3 pattern matching replaces long `if-else` and verbose `switch` blocks:

```
enum ExpenseCategory { food, transport, shopping, utilities }

// Expressive, concise Switch Expression returning a color token
Color getCategoryColor(ExpenseCategory category) => switch (category) {
  ExpenseCategory.food => Colors.orange,
  ExpenseCategory.transport => Colors.blue,
  ExpenseCategory.shopping => Colors.purple,
  ExpenseCategory.utilities => Colors.teal,
};

// Pattern matching on JSON payload
String describeTransaction(Map<String, dynamic> json) => switch (json) {
  {'type': 'expense', 'amount': double a} when a > 1000000 => "🚨 Major: $a VND",
  {'type': 'expense', 'amount': double a} => "Expense: $a VND",
  _ => "Other transaction",
};
```

Classes, Constructors & Parameter Patterns

Dart constructors provide elegant shorthands tailored for widget declarations:

```
class ExpenseItem {  
  final String id;  
  final String title;  
  final double amount;  
  final DateTime timestamp;  
  
  // Shorthand field initialization with required named parameters  
  const ExpenseItem({  
    required this.id,  
    required this.title,  
    required this.amount,  
    required this.timestamp,  
  });  
  
  // Named constructor with initializer list  
  ExpenseItem.draft(this.title, this.amount)  
    : id = "draft_${DateTime.now().millisecondsSinceEpoch}",  
      timestamp = DateTime.now();  
}
```

The Power of `const` Constructors

In Flutter, adding `const` to widgets is a critical performance optimization:

```
// ❌ Without const: Re-allocated in memory on EVERY parent rebuild
Widget buildBadgeBad() {
  return Container(
    padding: EdgeInsets.all(8.0),
    child: Text("Active", style: TextStyle(color: Colors.green)),
  );
}

// ✅ With const: Canonicalized in memory at compile-time (Zero GC pressure)
Widget buildBadgeGood() {
  return const Padding(
    padding: EdgeInsets.all(8.0),
    child: Text("Active", style: TextStyle(color: Colors.green)),
  );
}
```


Asynchronous Programming: Futures & Streams

Mobile apps interact constantly with async file I/O, camera sensors, and REST APIs:

```
// Future: Single value delivered in the future
Future<String> readReceiptFile(String path) async {
  final file = File(path);
  return await file.readAsString();
}

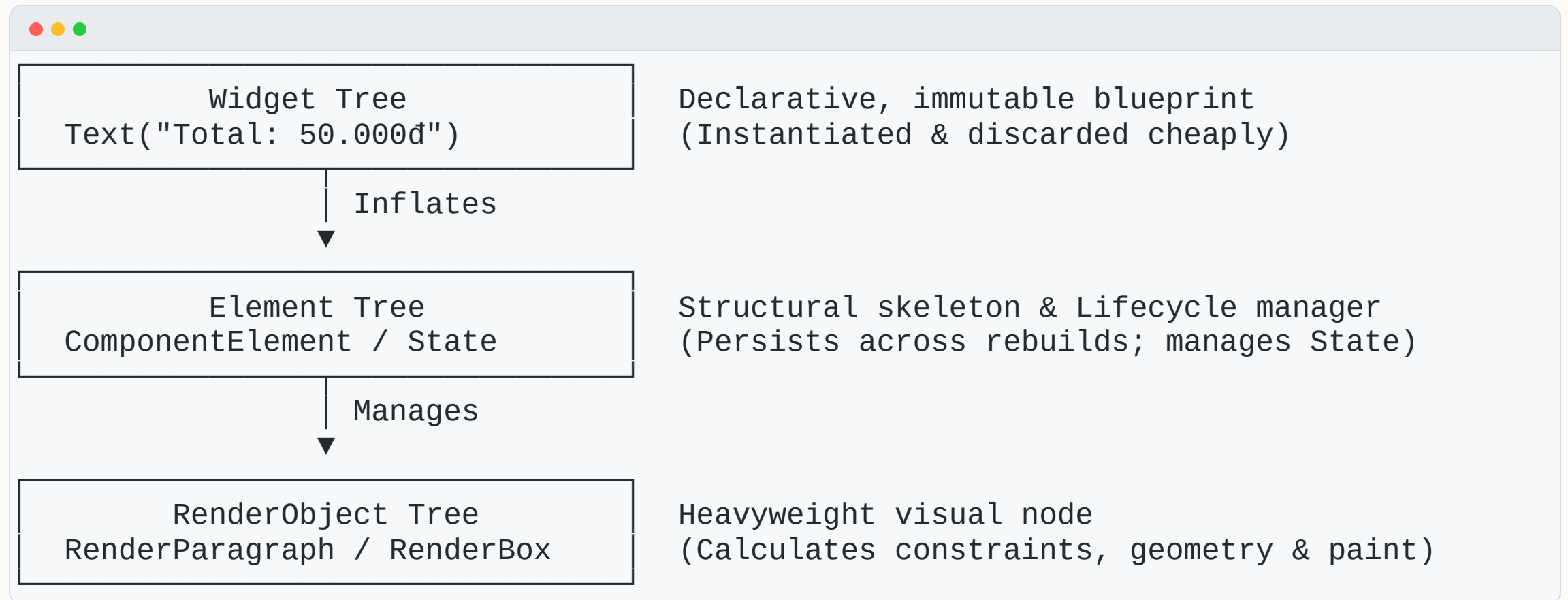
// Stream: Continuous sequence of asynchronous events (e.g. sensor readings)
Stream<int> countdownTimer(int seconds) async* {
  for (int i = seconds; i >= 0; i--) {
    await Future.delayed(const Duration(seconds: 1));
    yield i; // Emit next stream value
  }
}
```

Flutter provides `FutureBuilder` and `StreamBuilder` widgets to bind async data directly to UI.

Part 3: The 3 Trees Architecture

The 3 Trees in Flutter

Flutter does not render directly from your widget definitions. It orchestrates **3 parallel trees**:



Tree 1: The Widget Tree

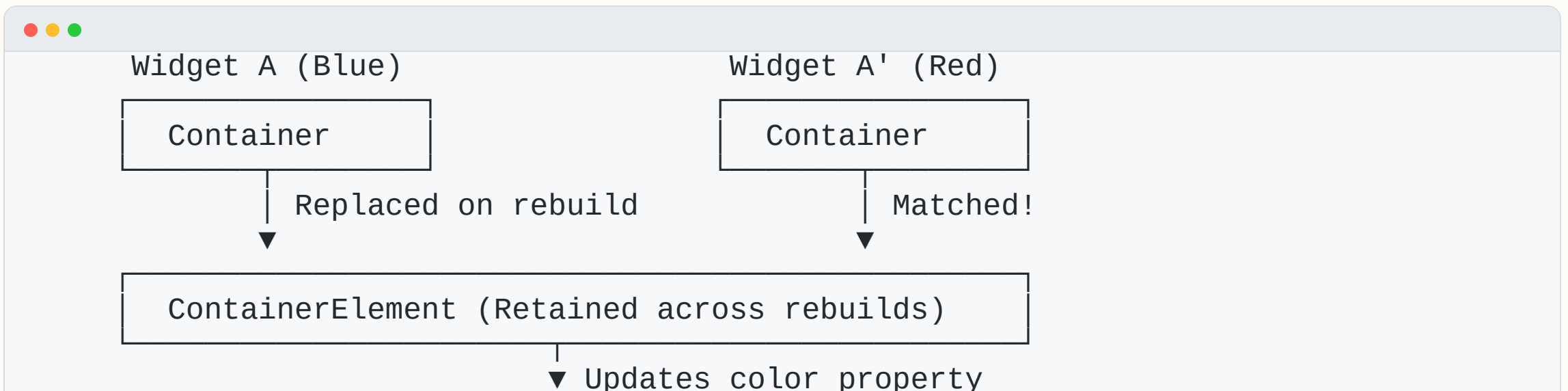
- **Nature:** Immutable, declarative, ultra-lightweight Dart objects.
- **Responsibility:** Describes the configuration and visual properties of the UI.
- **Lifecycle:** Created and garbage-collected constantly during every `build()`.
- **Cost:** Virtually free. Creating 10,000 widget instances takes under a few milliseconds because they are plain configuration references without layout or canvas memory.

```
// Every time setState() runs, new Widget instances are created:  
Container(  
  color: Colors.blue,  
  child: const Text("Scan Receipt"),  
)
```

Widgets are just the **blueprints**; they don't actually draw anything on the screen.

Tree 2: The Element Tree

- **Nature:** Persistent, mutable node that lives in memory as long as the widget remains on screen.
- **Responsibility:**
 - Holds references to the `Widget` configuration above it and the `RenderObject` below it.
 - Holds the mutable `State` instance for `StatefulWidget`.
 - Determines whether an existing `RenderObject` can be updated or must be recreated.



Tree 3: The RenderObject Tree

- **Nature:** Heavyweight, mutable C++/Dart objects that interface directly with the rendering pipeline.
- **Responsibility:**
 - **Layout:** Calculates box constraints (`minWidth` , `maxWidth` , `minHeight` , `maxHeight`) and computes final sizes and offsets.
 - **Painting:** Executes drawing commands on the Skia / Impeller `Canvas` .
 - **Hit Testing:** Routes touch events from screen coordinates to gesture recognizers.
- **Cost:** Expensive to allocate. Flutter reuses existing `RenderObject` nodes whenever possible.

Example: A `Text` widget creates a `StatelessElement` , which in turn manages a `RenderParagraph` render object.

How Flutter Rebuilds: `canUpdate()`

When `setState()` triggers a rebuild, Flutter calls static method `Widget.canUpdate()` :

```
static bool canUpdate(Widget oldWidget, Widget newWidget) {  
    return oldWidget.runtimeType == newWidget.runtimeType  
        && oldWidget.key == newWidget.key;  
}
```

- **If `true`** : The existing `Element` stays in place, points to the new widget, and updates its underlying `RenderObject` 's properties (e.g. text or color). **Extremely fast!**
- **If `false`** : The old `Element` and its entire `RenderObject` subtree are unmounted and disposed, and a completely fresh element is created.

The Role of **Key** in Flutter

When items in a dynamic list change position or are deleted, **runtimeType** alone cannot distinguish them:

```
// ✗ Without Keys: Reordering or deleting list items causes State to mismatch!  
ListView(  
  children: items.map((item) => ExpenseCard(data: item)).toList(),  
)  
  
// ✓ With GlobalKey: Element tree correctly matches State with unique item ID  
ListView(  
  children: items.map((item) => ExpenseCard(  
    key: GlobalKey(item.id),  
    data: item,  
  )).toList(),  
)
```

- **ValueKey(id)** : Matches elements by value equality (e.g. database ID).
- **ObjectKey(obj)** : Matches elements by object identity.

Part 4: Stateless vs. Stateful Widgets & Lifecycle

StatelessWidget : Pure Immutable UI

Use `StatelessWidget` when visual appearance depends solely on constructor inputs:

```
class ReceiptAmountBadge extends StatelessWidget {
  final double amount;
  final bool isHighValue;

  const ReceiptAmountBadge({
    super.key,
    required this.amount,
    this.isHighValue = false,
  });

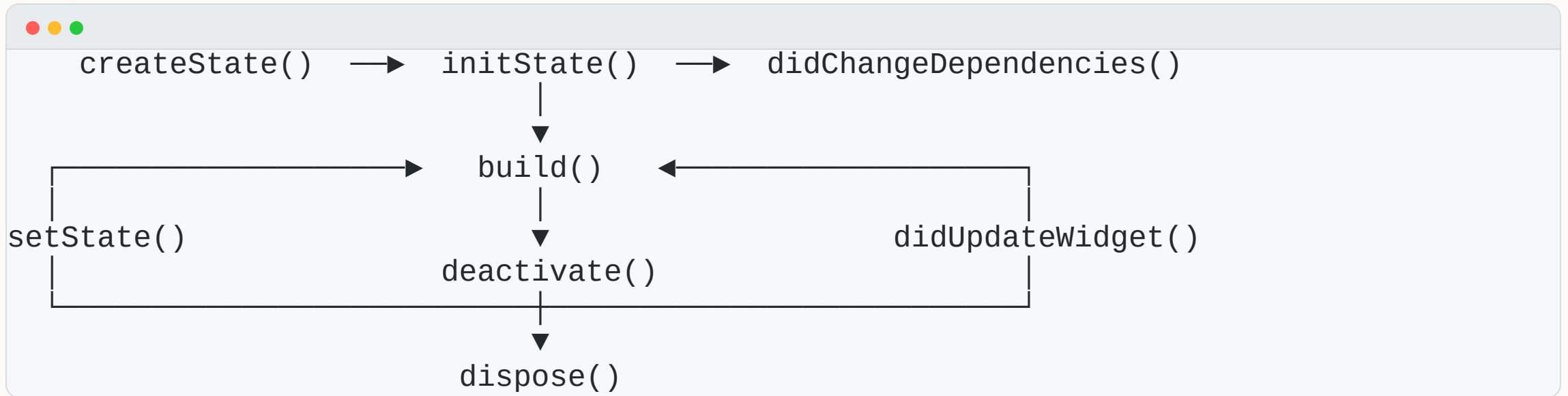
  @override
  Widget build(BuildContext context) {
    final color = isHighValue ? Colors.red.shade100 : Colors.green.shade100;
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
      decoration: BoxDecoration(color: color, borderRadius: BorderRadius.circular(12)),
      child: Text("${amount.toStringAsFixed(0)} đ"),
    );
  }
}
```

StatefulWidget : Managing Mutable State

When a widget needs to alter its appearance dynamically, split it into **two classes**:

```
class OCRScanCounter extends StatefulWidget {  
  final int limit;  
  const OCRScanCounter({super.key, this.limit = 10});  
  
  @override  
  State<OCRScanCounter> createState() => _OCRScanCounterState();  
}  
  
class _OCRScanCounterState extends State<OCRScanCounter> {  
  late int _remaining = widget.limit;  
  
  @override  
  Widget build(BuildContext context) {  
    return ElevatedButton(  
      onPressed: _remaining > 0 ? () => setState(() => _remaining--) : null,  
      child: Text("Scans Remaining: $_remaining"),  
    );  
  }  
}
```

The Complete State Lifecycle



- **`initState()`** : Called once when the element is created. Ideal for controller init.
- **`build()`** : Pure function called whenever the element is marked dirty.
- **`dispose()`** : Called when the element is removed permanently. Cancel timers here!

Critical Lifecycle Methods: `initState()` & `dispose()`

Mastering memory lifecycle is essential to prevent severe mobile memory leaks:

```
class ReceiptInputScreenState extends State<ReceiptInputScreen> {  
  final _amountController = TextEditingController(text: "0");  
  final _amountFocus = FocusNode();  
  
  @override  
  void dispose() {  
    // ⚠ ALWAYS dispose controllers and focus nodes to prevent memory leaks!  
    _amountController.dispose();  
    _amountFocus.dispose();  
    super.dispose();  
  }  
  
  @override  
  Widget build(BuildContext context) {  
    return TextField(controller: _amountController, focusNode: _amountFocus);  
  }  
}
```

Rebuild Optimization & Antipatterns

✗ Antipattern 1: Heavy Computation inside `build()`

```
@override
Widget build(BuildContext context) {
  // BAD: Heavy regex parsing executed on every single frame!
  final parsed = parseRawOcrFile();
  return Text(parsed.total);
}
```

Fix: Move parsing to `initState()` or a background Worker Isolate.

✗ Antipattern 2: Calling `setState()` High in the Tree

Rebuilding the entire screen when only an icon changes drops frame rates.

Fix: Encapsulate localized state in smaller child widgets or use state selectors.

✓ Best Practice: Extract Subtrees with `const`

Part 5: Layout Composition & Material Design 3

Material Design 3 (M3) Foundation

Modern Flutter apps use Material 3 by default with automatic seed color harmonization:

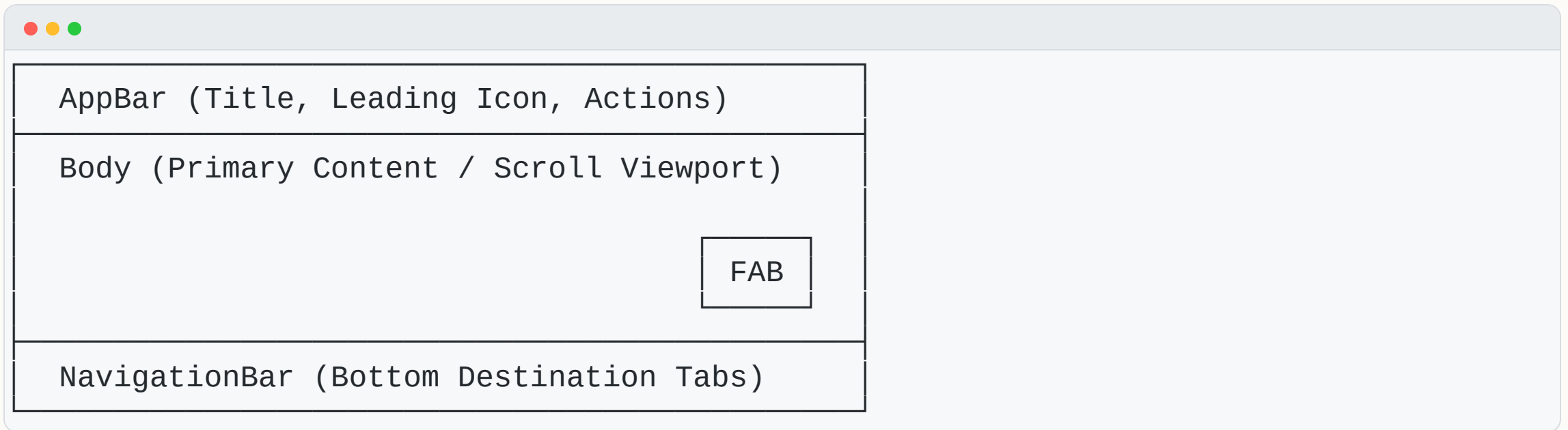
```
void main() => runApp(const ExpenseTrackerApp());

class ExpenseTrackerApp extends StatelessWidget {
  const ExpenseTrackerApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'VKU Expense OCR',
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF2C4570), // VKU Navy
        ),
      ),
      home: const ExpenseHomeScreen(),
    );
  }
}
```


Core Container: Scaffold Structure

`Scaffold` provides the fundamental Material 3 visual screen layout:



- Slots: `appBar`, `body`, `floatingActionButton`, `bottomNavigationBar`, `drawer`.

Scaffold Implementation

```
Scaffold(  
  appBar: AppBar(  
    title: const Text("My Receipts"),  
    actions: [IconButton(icon: const Icon(Icons.filter_list), onPressed: () {})],  
  ),  
  body: const Center(child: Text("No expenses scanned yet")),  
  floatingActionButton: FloatingActionButton.extended(  
    onPressed: () {},  
    icon: const Icon(Icons.camera_alt),  
    label: const Text("Scan Receipt"),  
  ),  
  bottomNavigationBar: NavigationBar(  
    destinations: const [  
      NavigationDestination(icon: Icon(Icons.receipt), label: "Expenses"),  
      NavigationDestination(icon: Icon(Icons.pie_chart), label: "Reports"),  
    ],  
  ),  
);
```

Flex Layouts: Row & Column

Flutter uses flexbox layout principles via **Row** (horizontal) and **Column** (vertical):

```
Row(  
  mainAxisAlignment: MainAxisAlignment.spaceBetween, // Horizontal spacing  
  crossAxisAlignment: CrossAxisAlignment.center,    // Vertical alignment  
  children: [  
    const Icon(Icons.storefront, color: Colors.blue),  
    const SizedBox(width: 8),  
    const Text("Highlands Coffee", style: TextStyle(fontWeight: FontWeight.bold)),  
    const Spacer(),  
    Text("65.000 đ", style: TextStyle(color: Colors.red.shade700)),  
  ],  
)
```

- **mainAxisAlignment** : Controls alignment along the primary axis (**start** , **center** , **end** , **spaceBetween**).
- **crossAxisAlignment** : Controls alignment along the perpendicular cross axis (**start** , **center** , **stretch**).

Flexible Sizing: Expanded vs. Flexible

Preventing the dreaded "RenderFlex overflowed by N pixels" error:

```
Row(  
  children: [  
    const CircleAvatar(child: Icon(Icons.receipt)),  
    const SizedBox(width: 12),  
    // Expanded forces child to consume ALL remaining space  
    Expanded(  
      child: Column(  
        crossAxisAlignment: CrossAxisAlignment.start,  
        children: const [  
          Text("Highlands Coffee Da Nang", overflow: TextOverflow.ellipsis),  
          Text("22/10/2026", style: TextStyle(color: Colors.grey)),  
        ],  
      ),  
    ),  
    IconButton(icon: const Icon(Icons.chevron_right), onPressed: () {}),  
  ],  
)
```

Overlays & Layering: **Stack** & **Positioned**

When UI elements must overlap (e.g. camera viewfinders, badges, floating cards):

```
Stack(
  alignment: Alignment.center,
  children: [
    // 1. Base Layer
    ClipRect(
      borderRadius: BorderRadius.circular(12),
      child: Image.network("https://picsum.photos/300/180", fit: BoxFit.cover),
    ),
    // 2. Positioned Badge pinned to top-right
    Positioned(
      top: 8,
      right: 8,
      child: Container(
        padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
        color: Colors.black54,
        child: const Text("OCR Verified", style: TextStyle(color: Colors.white, fontSize: 11)),
      ),
    ),
  ],
)
```

Virtualized Scrolling: `ListView.builder`

Never use `SingleChildScrollView(child: Column(...))` for long or dynamic database records!

```
//  High-Performance Virtualized Viewport (Recycles off-screen widgets)
ListView.builder(
  itemCount: expenses.length,
  itemBuilder: (BuildContext context, int index) {
    final item = expenses[index];
    return Dismissible(
      key: ValueKey(item.id),
      onDismissed: (dir) => deleteExpense(item.id),
      child: ListTile(
        leading: CircleAvatar(child: Text("${index + 1}")),
        title: Text(item.title),
        subtitle: Text(item.formattedDate),
        trailing: Text("${item.amount} đ"),
      ),
    );
  },
);
```

Device Boundaries: SafeArea & MediaQuery

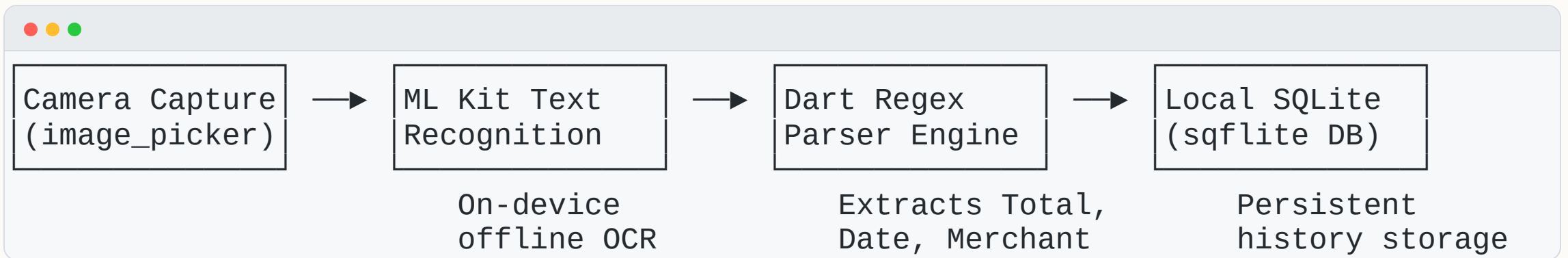
Adapting to notches, rounded screen corners, dynamic islands, and screen orientations:

```
Widget build(BuildContext context) {  
  final mediaQuery = MediaQuery.of(context);  
  final isPortrait = mediaQuery.orientation == Orientation.portrait;  
  
  return Scaffold(  
    // SafeArea automatically inserts padding to clear hardware notches & home bars  
    body: SafeArea(  
      child: Padding(  
        padding: const EdgeInsets.all(16.0),  
        child: Column(  
          children: [  
            Text("Screen Width: ${mediaQuery.size.width.toInt()} px"),  
            Text("Orientation: ${isPortrait ? 'Portrait' : 'Landscape'}"),  
          ],  
        ),  
      ),  
    ),  
  );  
}
```

Part 6: Mini-Project 3 Kickoff

Mini-Project 3: Receipt OCR & Expense Tracker

Scenario: VKU students and club managers deal with cash receipts daily. Manually typing expense numbers into spreadsheets is tedious and error-prone.



Key Deliverable: A production-ready Flutter app running on physical mobile devices capable of scanning receipts, extracting totals via ML Kit, verifying data on review screens, and drawing custom canvas charts.

Project Specifications & 10-Point Rubric

Component	Target Capabilities	Points
On-Device OCR & Heuristics	Camera capture, ML Kit text recognition, Regex extraction of Total amount, Date, and Merchant header	3.5 pts
Custom Canvas Visualization	Custom-drawn animated Pie/Donut category chart & Weekly bar chart using <code>CustomPainter</code>	2.5 pts
State Management & DB	Clean <code>Provider</code> or <code>Riverpod 2</code> architecture, persistent <code>sqflite</code> CRUD, receipt photo storage	2.0 pts
UI/UX Polish	Material 3 themes, dark mode, responsive layout, review & manual correction dialog	1.0 pt
Deliverables & Report	Signed Release APK (<code>app-release.apk</code>), demo video, clean GitHub repository, 2–4 page technical PDF	1.0 pt

Project Setup & pubspec.yaml

```
name: vku_expense_ocr
environment:
  sdk: '>=3.0.0 <4.0.0'

dependencies:
  flutter:
    sdk: flutter
  google_mlkit_text_recognition: ^0.11.0
  image_picker: ^1.0.7
  flutter_riverpod: ^2.5.1
  go_router: ^13.2.0
  sqflite: ^2.3.2
  path: ^1.9.0
  intl: ^0.19.0
```

Part 7: In-Class Hands-on Lab & Week 7 Takeaways

In-Class Lab Exercise: Expense Summary Card (30 min)

Task: Create a reusable `ExpenseSummaryCard` widget incorporating:

1. An icon inside a circular container indicating category.
2. Store name and date stacked vertically with `CrossAxisAlignment.start`.
3. Highlighted monetary amount formatted as Vietnamese Dong (`###.### đ`).
4. Wrap inside a Material 3 `Card` with elevation and ink ripple `InkWell` tap callback.

```
class ExpenseSummaryCard extends StatelessWidget {  
  final String merchant;  
  final double amount;  
  final DateTime date;  
  final VoidCallback onTap;  
  
  const ExpenseSummaryCard({  
    super.key,  
    required this.merchant,  
    required this.amount,  
    required this.date,
```

Week 7 Key Takeaways

- **✓ The Declarative Paradigm:** UI is a pure function of state ($UI = f(state)$).
- **✓ Zero Bridge:** Dart compiles to native machine code via AOT, drawn at 120Hz via the modern Impeller engine.
- **✓ The 3 Trees:** The lightweight Widget tree dictates configuration; the persistent Element tree orchestrates State and lifecycle; the RenderObject tree calculates layout geometry and paints pixels.
- **✓ Lifecycle Safety:** Always dispose controllers, focus nodes, and event listeners in `dispose()` to prevent memory leaks.
- **✓ Composition Over Inheritance:** Assemble screens by composing primitives: `Scaffold`, `Row`, `Column`, `Expanded`, `Stack`, and `ListView.builder`.

Next Week (Week 8): State Management deep-dive (Provider, Riverpod 2, BLoC), Declarative Routing with GoRouter, Form Validation, and Custom Canvas Drawing with `CustomPainter` !